

Java 8 Streams — Interview Question Bank (with Answers)

1. Stream Basics & Fundamentals

Q: What is a Stream in Java 8? How is it different from a Collection? A Stream is a sequence of elements supporting functional-style, pipelined operations over a data source. Unlike a Collection, it stores no elements itself, is lazily evaluated, can be infinite, and is consumed once by a terminal operation. A Collection is an in-memory data structure you can iterate repeatedly; a Stream is a computation over a source.

Q: Key characteristics of a Stream? Not a data structure; functional (doesn't mutate the source); lazy (intermediate ops don't execute until a terminal op runs); can represent infinite sequences; traversable only once.

Q: Difference between intermediate and terminal operations? Intermediate operations (filter, map, sorted, ...) return a new Stream and are lazy — merely recorded into the pipeline. Terminal operations (collect, forEach, reduce, ...) trigger execution of the whole pipeline and produce a result or side effect, after which the stream is consumed.

Q: Why are intermediate operations lazy? What triggers execution? Laziness lets the JDK fuse all operations into a single pass per element and short-circuit early (e.g., stop after findFirst()), avoiding unnecessary work and unnecessary intermediate collections. Execution is triggered only when a terminal operation is invoked.

Q: Can a stream be reused/traversed twice? No. Once a terminal operation runs, the stream is considered closed. Reusing it throws IllegalStateException: stream has already been operated upon or closed.

Q: What is the Stream pipeline? source.stream() → intermediate op 1 → intermediate op 2 → ... → terminal op. Nothing executes until the terminal op is called, at which point each element flows through the whole chain.

Q: What functional interfaces back common stream operations? Predicate<T> (filter), Function<T,R> (map), Consumer<T> (forEach/peek), Supplier<T> (generate), BinaryOperator<T> (reduce), UnaryOperator<T> (iterate), Comparator<T> (sorted).

Q: Stream vs IntStream/LongStream/DoubleStream — why do primitive streams exist? IntStream/LongStream/DoubleStream operate on primitives directly, avoiding the boxing/unboxing overhead of Stream<Integer> etc., and they expose numeric terminal ops like sum(), average(), summaryStatistics() that a generic Stream<T> doesn't have.

Q: Is a Stream thread-safe? Can multiple threads use the same instance? A Stream instance itself isn't designed for concurrent external use — you shouldn't call operations on the same stream from multiple threads. Parallel streams handle internal parallelism safely on your behalf, provided the lambdas you supply are stateless and non-interfering.

Q: What happens if you call a terminal operation twice? The second call throws `IllegalStateException` because the stream was already consumed by the first terminal operation.

2. Creating Streams

Q: Ways to create a Stream: `collection.stream()`, `Stream.of(a, b, c)`, `Arrays.stream(array)`, `Stream.generate(Supplier)`, `Stream.iterate(seed, UnaryOperator)`, `IntStream.range()/rangeClosed()`, `Files.lines(path)`, `pattern.splitAsStream(str)`.

Q: `Stream.generate()` vs `Stream.iterate()`? `generate(Supplier)` produces each element independently of prior ones (e.g., random values, constants). `iterate(seed, f)` produces each element by applying `f` to the previous element, forming a dependent sequence (e.g., powers of two). Both are infinite unless bounded with `limit()`.

Q: How do you create an infinite stream and safely limit it? `Stream.iterate(1, n -> n * 2).limit(10)` or `Stream.generate(Math::random).limit(5)` — `limit()` is short-circuiting, so it stops pulling more elements once satisfied.

Q: `IntStream.range()` vs `rangeClosed()`? `range(a, b)` is exclusive of `b`; `rangeClosed(a, b)` is inclusive of `b`.

Q: Create a Stream from an array vs a List vs a Map? Array: `Arrays.stream(arr)`. List: `list.stream()`. Map: no direct `.stream()` — use `map.entrySet().stream()` (or `.keySet().stream()` / `.values().stream()`).

Q: Create a Stream of characters from a String? `"hello".chars()` returns an `IntStream` of char codes; map with `c -> (char) c` or collect via `.mapToObj(c -> (char) c)` to get a `Stream<Character>`.

Q: Convert a Stream to an array and back? To array: `stream.toArray(String[]::new)`. From array back to stream: `Arrays.stream(array)`.

Q: Create an empty Stream — when is it useful? `Stream.empty()`. Useful as a safe return value from a method instead of null, or as the identity in `flatMap/Stream.concat` chains.

Q: Create a Stream from multiple existing streams? `Stream.concat(stream1, stream2)` (for more than two, chain concat calls or `flatMap` over a stream of streams).

Q: `Stream.of(array)` vs `Arrays.stream(array)` for object vs primitive arrays? For object arrays, both behave the same (`Stream.of(objArray)` delegates to `Arrays.stream`). For primitive arrays (e.g., `int[]`), `Stream.of(intArray)` produces a single-element `Stream<int[]>` (the whole array as one element), whereas `Arrays.stream(intArray)` correctly produces an `IntStream` of the individual elements.

3. Intermediate Operations

Q: Explain `filter()` with an example. `list.stream().filter(n -> n % 2 == 0)` keeps only elements matching the `Predicate<T>`.

Q: map() vs flatMap()? When to use flatMap()? map() transforms each element 1-to-1 (`Function<T,R>`). `flatMap()` transforms each element into a `Stream` and flattens all resulting streams into one — used when each element itself represents a collection/stream of values you want merged into a single flat stream (e.g., `List<List<Integer>>` → `List<Integer>`).

Q: Flatten a List<List> into a single List:

```
List<Integer> flat = nested.stream()
    .flatMap(List::stream)
    .collect(Collectors.toList());
```

Q: What does distinct() rely on internally? `equals()` and `hashCode()` of the element type.

Q: How does sorted() work with natural ordering vs a Comparator? `sorted()` requires elements to implement `Comparable` and uses `compareTo()`. `sorted(Comparator<T>)` uses the supplied comparator instead, allowing custom or multi-field ordering.

Q: Explain peek() and why it's mainly for debugging. `peek(Consumer)` runs a side-effect action on each element without altering the stream's contents, and returns the same stream for further chaining. It shouldn't drive business logic because, due to laziness/short-circuiting, it may not execute for every element, and its ordering isn't guaranteed under parallel execution — it's intended purely for inspection/logging.

Q: limit() vs skip()? Pagination with streams? `limit(n)` keeps only the first `n` elements; `skip(n)` discards the first `n`. Pagination: `stream.skip((page - 1) * pageSize).limit(pageSize)`.

Q: Chain multiple map()/filter()? Does order matter for performance? Yes, you can chain freely. Order can matter for performance — filtering before an expensive `map()` reduces the number of elements the mapper has to process, so put cheaper/selective filters earlier in the chain when possible.

Q: map() vs mapToInt()/mapToObj()? map() stays within `Stream<T>` (boxed). `mapToInt()/mapToLong()/mapToDouble()` convert a `Stream<T>` into a primitive stream. `mapToObj()` converts a primitive stream back into a `Stream<T>` (boxed objects).

Q: How do intermediate ops avoid eager processing? Because they're lazily composed into a single pipeline that's only walked when a terminal op runs, and short-circuiting operations like `limit()/findFirst()` stop pulling elements as soon as the result is determined — so, for example, `stream.filter(...).findFirst()` may only touch a handful of elements even in a huge or infinite source.

4. Terminal Operations

Q: List common terminal operations. `forEach()`, `collect()`, `reduce()`, `count()`, `anyMatch()/allMatch()/noneMatch()`, `findFirst()/findAny()`, `toArray()`, `min()/max()`.

Q: Explain reduce()'s three overloads. 1. reduce(BinaryOperator<T>) → Optional<T> — no seed; empty stream gives empty Optional. 2. reduce(T identity, BinaryOperator<T>) → T — identity is both the seed and the result for an empty stream. 3. reduce(U identity, BiFunction<U,T,U> accumulator, BinaryOperator<U> combiner) → U — used when accumulating into a different type than the stream's element type.

Q: When is the combiner in reduce() actually used? Only in parallel execution, to merge partial results computed independently on different chunks/threads. In sequential streams, the combiner is never invoked.

Q: findFirst() vs findAny(), especially in parallel? findFirst() always returns the first element in encounter order, even for parallel streams (which limits parallel speed-up). findAny() is unconstrained by order and can return as soon as any thread finds a match, making it faster in parallel but non-deterministic across runs for streams with multiple matches.

Q: forEach() vs forEachOrdered()? forEach() doesn't guarantee encounter order in a parallel stream. forEachOrdered() processes elements in encounter order regardless of parallelism, at the cost of giving up most parallel performance benefit.

Q: anyMatch/allMatch/noneMatch on an empty stream? anyMatch → false; allMatch → true; noneMatch → true (vacuously true, since there's nothing to violate the condition).

Q: Why does a terminal op make the stream unusable afterward? Because the pipeline's internal state is consumed while iterating the source once; the JDK enforces "operate once" semantics and throws IllegalStateException on reuse rather than silently producing wrong or partial results.

Q: What is short-circuiting? Which ops are short-circuiting? An operation that can produce a result (or terminate) without processing every element. Short-circuiting ops: limit, findFirst, findAny, anyMatch, allMatch, noneMatch.

Q: Count elements matching a condition without collecting to a list first? long count = stream.filter(predicate).count(); — no intermediate list needed.

Q: Terminal operation on an already-consumed stream? Throws IllegalStateException.

5. Collectors & Collecting Results

Q: toList(), toSet(), toMap() — duplicate keys in toMap()? toList()/toSet() collect into a List/Set. toMap(keyFn, valueFn) throws IllegalStateException if two elements produce the same key, because it doesn't know how to resolve the collision without a merge function.

Q: Provide a merge function to toMap()? toMap()

```
Map<String, Integer> m = list.stream()
    .collect(Collectors.toMap(Employee::getDept, Employee::getSalary, (v1, v2) ->
v1 + v2));
```

Q: groupingBy() with a single classifier, and with a downstream collector?

groupingBy(Employee::getDept) → Map<String, List<Employee>>. With a downstream: groupingBy(Employee::getDept, Collectors.counting()) → Map<String, Long>.

Q: Multi-level (nested) groupingBy?

```
Map<String, Map<String, List<Employee>>> byDeptThenTitle =
employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.groupingBy(Employee::getTitle)));
```

Q: partitioningBy() vs groupingBy()? partitioningBy(predicate) always produces exactly two groups keyed by Boolean (true/false), even if one group is empty. groupingBy(classifier) produces an arbitrary number of groups keyed by the classifier's result.

Q: joining() and its overloads? Collectors.joining() concatenates with no separator; joining(delimiter) inserts a separator between elements; joining(delimiter, prefix, suffix) also wraps the whole result, e.g. "[a, b, c]".

Q: counting(), summingInt/Double(), averagingInt(), summarizingInt()? counting() → element count as Long. summingInt/Double() → sum. averagingInt() → average as Double. summarizingInt() → an IntSummaryStatistics object with count, sum, min, average, and max all at once.

Q: Collectors.mapping() with groupingBy()? mapping(function, downstream) transforms each element before it reaches a downstream collector — commonly used to project one field per group:

```
Map<String, List<String>> namesByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.mapping(Employee::getName, Collectors.toList())));
```

Q: Collectors.reducing() vs stream.reduce()? reducing() is a Collector-based equivalent of reduce(), mainly used as a *downstream* collector inside groupingBy/partitioningBy where you need reduction per group, since those APIs expect a Collector, not a plain reduce call.

Q: Collect into an immutable list/map? Collectors.toUnmodifiableList() / toUnmodifiableMap(), or wrap any collector: Collectors.collectingAndThen(Collectors.toList(), Collections::unmodifiableList).

Q: Write a custom Collector using Collector.of().

```
Collector<String, StringBuilder, String> customJoiner = Collector.of(
    StringBuilder::new,
```

```
(sb, s) -> sb.append(s).append(","),
StringBuilder::append,
StringBuilder::toString
);
```

Q: Three main components of a Collector, plus optional finisher/characteristics?

Supplier (creates the mutable container), accumulator (folds an element in), combiner (merges two containers, used in parallel), and an optional finisher (transforms the container into the final result) plus characteristics that describe optimizable behavior.

Q: Collector characteristics — CONCURRENT, UNORDERED, IDENTITY_FINISH?

CONCURRENT — the accumulator can be safely called concurrently on a shared container from multiple threads (enables a parallel optimization). UNORDERED — the collector doesn't care about encounter order. IDENTITY_FINISH — the finisher is the identity function, so the JDK can skip calling it and return the container directly.

6. Optional

Q: What is Optional and what problem does it solve? A container object that may or may not hold a non-null value, used mainly as a return type to make “this might be absent” explicit in the API, reducing accidental NullPointerExceptions and unnecessary null checks.

Q: Optional.of() vs ofNullable() vs empty()? of(value) throws NullPointerException if value is null. **ofNullable(value)** safely wraps a possibly-null value (empty if null). **empty()** explicitly creates an empty Optional.

Q: orElse() vs orElseGet() vs orElseThrow()? orElse(default) always evaluates default eagerly, even when the Optional has a value. **orElseGet(Supplier)** only invokes the supplier when the Optional is empty (lazy). **orElseThrow(...)** throws a given (or default NoSuchElementException) if empty.

Q: Why prefer orElseGet() for expensive defaults? Because **orElse()**'s argument is computed unconditionally at the call site regardless of whether it's needed, wasting work when the Optional is present; **orElseGet()** defers that computation to only when actually required.

Q: map()/flatMap() on Optional? **map(Function)** transforms the contained value if present, else stays empty. **flatMap(Function)** is used when the mapper itself returns an Optional, to avoid nesting into `Optional<Optional<T>>`.

Q: How do findFirst()/findAny()/min()/max() relate to Optional? They all return `Optional<T>` because the stream (or a filtered subset of it) might be empty, so there may be no result to return.

Q: Optional as a method parameter or field? Generally discouraged — Optional is designed as a return type communicating “may be absent” to callers. Using it as a parameter/field adds overhead (it's not Serializable, and it's another layer of indirection) without real benefit; a nullable parameter or overloaded methods are usually clearer.

Q: Convert an Optional to a Stream? `optional.stream()` (Java 9+) returns a `Stream` of 0 or 1 elements — useful for flat-mapping a `Stream<Optional<T>>` into a `Stream<T>` of only present values.

Q: Avoid `isPresent()` + `get()` anti-pattern? Use `ifPresent(Consumer)` to run an action only if present, or `ifPresentOrElse(Consumer, Runnable)` to also handle the empty case, instead of manually checking then unwrapping.

7. Parallel Streams

Q: How do you create a parallel stream? `collection.parallelStream()`, or convert an existing stream with `.parallel()`.

Q: How does Fork/Join power parallel streams? The source is recursively split into chunks (via `Splitter`), each chunk is processed on a worker thread from `ForkJoinPool.commonPool()` using the fork/join divide-and-conquer algorithm, and partial results are merged back together (using the collector's combiner or `reduce`'s combiner).

Q: `ForkJoinPool.commonPool()` — default thread count?

`Runtime.getRuntime().availableProcessors()` - 1 worker threads by default.

Q: When should you avoid parallel streams? Small datasets (overhead outweighs benefit), I/O-bound or blocking work (threads sit idle, starving the shared pool), poorly-splitting sources (`LinkedList`, `Stream.iterate()`), and any workload relying on ordering or touching shared mutable state.

Q: Why are `peek()`/`forEach()` side effects dangerous in parallel streams? Because there's no synchronization guarantee — if the side effect mutates shared state without external synchronization, you get race conditions and non-deterministic/incorrect results.

Q: `findFirst()` vs `findAny()` performance in parallel? `findAny()` can return as soon as any thread finds a match, so it's typically faster in parallel. `findFirst()` must preserve encounter order, so it may need to wait on earlier chunks even if a later chunk matches sooner — costing some parallel speed-up.

Q: Why must accumulator/combiner be associative and stateless? Because the stream may be split arbitrarily into chunks processed independently and merged in any order; only associative, stateless functions guarantee the same final result regardless of how the data was partitioned and combined.

Q: Control parallelism level with a custom pool?

```
ForkJoinPool customPool = new ForkJoinPool(4);  
customPool.submit(() -> list.parallelStream().map(...).collect(...)).get();
```

Q: Ordered vs unordered parallel streams for `limit()`? An ordered parallel stream must respect encounter order for `limit()`, which can force extra coordination/buffering across threads. Calling `.unordered()` before `limit()` lets the JDK short-circuit more efficiently since any `n` elements will do, often improving parallel performance.

Q: Scenario where a parallel stream hurt performance. Example:

`IntStream.rangeClosed(1, 100).parallel().sum()` — for only 100 elements, the overhead of splitting work and coordinating threads exceeds the trivial cost of summing sequentially, so the parallel version is slower than a plain sequential stream or loop.

8. Method References & Functional Interfaces

Q: Four types of method references? 1. Static: `ClassName::staticMethod` 2. Instance method of a particular object: `instance::instanceMethod` 3. Instance method of an arbitrary object of a type: `ClassName::instanceMethod` 4. Constructor: `ClassName::new`

Q: Example of each, used in a stream pipeline.

```
list.stream().map(Integer::parseInt);           // static
list.stream().forEach(System.out::println);     // particular object
list.stream().map(String::toUpperCase);        // arbitrary object of a type
list.stream().map(Employee::new);              // constructor
```

Q: Function<T,R> vs BiFunction<T,U,R> vs UnaryOperator? `Function<T,R>` takes one argument and returns a (possibly different type) result. `BiFunction<T,U,R>` takes two arguments. `UnaryOperator<T>` is a `Function<T,T>` where input and output are the same type.

Q: Predicate.and()/or()/negate()? Compose boolean conditions: `isAdult.and(isActive)`, `isAdmin.or(isModerator)`, `isEmpty.negate()` — used to build complex filter conditions without writing a single big lambda.

Q: Function.andThen() vs compose()? `f.andThen(g)` applies `f` first, then `g` to its result. `f.compose(g)` applies `g` first, then `f` to its result — the order of execution is reversed relative to the method call order.

Q: Comparator.comparing(), thenComparing(), reversed()? `comparing(keyExtractor)` builds a comparator from a key. `thenComparing(...)` adds a tie-breaking comparator. `reversed()` inverts the ordering — used together for multi-field sorts.

Q: Sort by multiple fields using Comparator chaining.

```
list.stream()
    .sorted(Comparator.comparing(Employee::getDept)
        .thenComparing(Employee::getSalary, Comparator.reverseOrder()))
    .collect(Collectors.toList());
```

9. Common Coding Problems — Solutions

Find duplicate elements in a List:

```
Set<Integer> seen = new HashSet<>();
List<Integer> duplicates = list.stream()
    .filter(n -> !seen.add(n))
    .collect(Collectors.toList());
```

First non-repeated character in a String:

```
Character firstNonRepeated = str.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, LinkedHashMap::new,
Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() == 1)
    .map(Map.Entry::getKey)
    .findFirst()
    .orElse(null);
```

Frequency of each character/word in a String:

```
Map<String, Long> wordFreq = Arrays.stream(str.split("\\s+"))
    .collect(Collectors.groupingBy(w -> w, Collectors.counting()));
```

Highest-paid employee per department:

```
Map<String, Optional<Employee>> topPaidByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.maxBy(Comparator.comparing(Employee::getSalary))));
```

Average salary per department:

```
Map<String, Double> avgSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.averagingDouble(Employee::getSalary)));
```

Second highest number in a List:

```
Integer secondHighest = list.stream()
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst()
    .orElse(null);
```

Sort a Map<String, Integer> by its values:

```
Map<String, Integer> sorted = map.entrySet().stream()
    .sorted(Map.Entry.comparingByValue())
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
        (a, b) -> a, LinkedHashMap::new));
```

List to comma-separated String:

```
String csv = list.stream().collect(Collectors.joining(", "));
```

Remove duplicate objects by a custom field:

```
Collection<Employee> distinctByName = employees.stream()
    .collect(Collectors.toMap(Employee::getName, e -> e, (a, b) -> a,
```

```
LinkedHashMap::new))  
    .values();
```

Merge two lists into one without duplicates:

```
List<Integer> merged = Stream.concat(list1.stream(), list2.stream())  
    .distinct()  
    .collect(Collectors.toList());
```

Sum, average, min, max of a list of numbers:

```
IntSummaryStatistics stats =  
list.stream().mapToInt(Integer::intValue).summaryStatistics();  
// stats.getSum(), getAverage(), getMin(), getMax()
```

List to Map<Integer, String> of id-to-name:

```
Map<Integer, String> idToName = employees.stream()  
    .collect(Collectors.toMap(Employee::getId, Employee::getName));
```

Partition numbers into even and odd:

```
Map<Boolean, List<Integer>> partitioned = numbers.stream()  
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

Flatten List<List> into a sorted, distinct List:

```
List<String> result = nested.stream()  
    .flatMap(List::stream)  
    .distinct()  
    .sorted()  
    .collect(Collectors.toList());
```

Prime numbers up to N:

```
List<Integer> primes = IntStream.rangeClosed(2, n)  
    .filter(num -> IntStream.rangeClosed(2, (int) Math.sqrt(num)).allMatch(d ->  
num % d != 0))  
    .boxed()  
    .collect(Collectors.toList());
```

Reverse a String using streams:

```
String reversed = Arrays.stream(str.split(""))  
    .reduce("", (a, b) -> b + a);
```

Check if a String is a palindrome:

```
boolean isPalindrome = IntStream.range(0, str.length() / 2)  
    .allMatch(i -> str.charAt(i) == str.charAt(str.length() - 1 - i));
```

Longest word in a List:

```
String longest = list.stream()
    .max(Comparator.comparingInt(String::length))
    .orElse("");
```

Upper-case an array of Strings and collect into a List:

```
List<String> upper = Arrays.stream(arr)
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

Sum of all elements in a List<List>:

```
int total = nested.stream()
    .flatMap(List::stream)
    .reduce(0, Integer::sum);
```

First N Fibonacci numbers using Stream.iterate():

```
Stream.iterate(new int[]{0, 1}, f -> new int[]{f[1], f[0] + f[1]})
    .limit(n)
    .map(f -> f[0])
    .collect(Collectors.toList());
```

Comma-separated names sorted by salary descending:

```
String names = employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .map(Employee::getName)
    .collect(Collectors.joining(", "));
```

Count strings starting with a particular letter:

```
long count = list.stream().filter(s -> s.startsWith("A")).count();
```

Common elements between two lists:

```
List<Integer> common = list1.stream()
    .filter(list2::contains)
    .distinct()
    .collect(Collectors.toList());
```

Map<Department, List> then Map<Department, Long> of counts:

```
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept));

Map<String, Long> countByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept, Collectors.counting()));
```

10. Tricky / Conceptual Interview Questions

Q: Why doesn't filter().findFirst() process the entire stream? Because findFirst() is short-circuiting: the pipeline pulls elements one at a time from the source and stops as

soon as the filter predicate matches, instead of first materializing every filtered result and then taking the first one.

Q: Why avoid modifying an external variable inside a lambda in `forEach()/map()`? It breaks the functional, side-effect-free contract streams are designed around, isn't safe under parallel execution (race conditions), and Java doesn't even allow it directly unless the variable is effectively final (so you'd need a mutable wrapper, which is a code smell). Use `collect/reduce` to build results instead.

Q: Exception from modifying a collection while streaming over it? `ConcurrentModificationException`, because the stream's underlying iterator detects the structural modification (via a mod-count check) mid-traversal.

Q: Why can't you use a non-effectively-final local variable in a lambda? Because the lambda may be invoked later or on another thread, after the enclosing method's stack frame is gone; Java captures local variables by value at lambda-creation time, and allowing later reassignment would create inconsistent, hard-to-reason-about behavior — so the compiler requires the variable to never change after initialization.

Q: Performance: for-loop vs stream for simple iteration? For simple iteration over small/medium data, a plain for-loop is often as fast or faster because it avoids the overhead of lambda dispatch and pipeline setup. Streams tend to pay off more with complex chained transformations, parallelism, or when readability/composability outweighs micro-optimization; for hot, performance-critical loops, benchmark before assuming streams are faster.

Q: Autoboxing/unboxing cost: `Stream` vs `IntStream`? `Stream<Integer>` boxes every primitive into an Integer object and unboxes for arithmetic, which adds allocation and GC pressure. `IntStream` (and `LongStream/DoubleStream`) work with primitives directly, avoiding that cost — preferred for numeric-heavy pipelines.

Q: Why does `toMap()` throw `IllegalStateException` on duplicate keys, and the fix? Because with no merge strategy specified, the collector can't decide which value should win for a colliding key, so it fails fast rather than silently dropping data. Fix: supply a merge function, `toMap(keyFn, valueFn, (v1, v2) -> v1)`.

Q: `sorted()` on an infinite stream without `limit()`? `sorted()` is a stateful operation that must see every element before it can produce any output, so calling it on an infinite stream without first bounding it (e.g., with `limit()`) causes the pipeline to hang indefinitely (or eventually throw `OutOfMemoryError` trying to buffer everything).

Q: How to debug a stream pipeline that isn't producing expected results? Insert `peek(System.out::println)` between stages to inspect elements as they flow through each step, temporarily, to isolate which operation is misbehaving. Remove it once done — it's a debugging aid, not for production logic.

Q: Eager (Collections) vs lazy (Streams) evaluation, with a side-effect example?

```
List<Integer> list = List.of(1, 2, 3);
Stream<Integer> s = list.stream().map(n -> { System.out.println("mapping " + n);
return n * 2; });
// nothing printed yet — lazy
s.collect(Collectors.toList());
// now prints "mapping 1", "mapping 2", "mapping 3" during the terminal op
```

A Collection-based loop would print immediately as each element is processed, in an eager, imperative pass.

Q: Checked exceptions inside a lambda in a stream? Functional interfaces like Function/Consumer don't declare throws, so a checked exception inside a lambda must be caught and wrapped in an unchecked exception (e.g., RuntimeException), handled via a try/catch inside the lambda, or routed through a custom functional interface/helper method that performs the sneaky-throw or wraps it.

Q: Mutability of Collectors.toList() across JDK versions? In Java 8, Collectors.toList() happens to return a mutable ArrayList, but this isn't part of its documented contract. Java 16 introduced Stream.toList(), which explicitly returns an **immutable** list — attempting to modify it throws UnsupportedOperationException.

Q: Convert a Stream back into a legacy Iterator or Iterable? Iterator<T> it = stream.iterator(); or Iterable<T> iterable = stream::iterator; (using a method reference to adapt the Stream's iterator() to the Iterable functional interface).

Q: Implement a simplified map()/filter() to understand internals?

```
static <T, R> List<R> myMap(List<T> input, Function<T, R> fn) {
    List<R> result = new ArrayList<>();
    for (T t : input) result.add(fn.apply(t));
    return result;
}

static <T> List<T> myFilter(List<T> input, Predicate<T> pred) {
    List<T> result = new ArrayList<>();
    for (T t : input) if (pred.test(t)) result.add(t);
    return result;
}
```

Real Streams do this lazily, chaining operations into a single pass via internal Sink objects, rather than materializing a new list at every step as this simplified version does.

Q: What design pattern do Streams follow internally? The pipeline is built from a chain of Sink objects, each wrapping the next — conceptually similar to the **Decorator pattern** — so that when the terminal operation drives iteration, each element flows through every stage's accept() call in a single pass, rather than each intermediate operation eagerly producing a whole new collection.